

Programowanie obiektowe

Wykład 4

Przeciążanie składowych

Przeciążanie składowych (ang. *Overloading*)

Przeciążanie składowych to mechanizm programowania obiektowego, który pozwala na definiowanie w obrębie jednej klasy wielu funkcji (metod, operatorów lub konstruktorów) o tej samej nazwie, ale z różnymi sygnaturami. Oznacza to, że funkcje różnią się **liczbą, typem lub kolejnością parametrów**, natomiast mogą zwracać wartość tego samego lub innego typu.

Kluczowe cechy:

- Ta sama nazwa funkcji/metody/operatora
- Różna lista parametrów (liczba, typ, kolejność)
- Rozstrzygane na etapie **kompilacji** (*polimorfizm statyczny*)
- Nie można przeciążać wyłącznie na podstawie zwracanego typu

Rodzaje przeciążania:

- **Metod** – różne zestawy parametrów tej samej funkcji
- **Operatorów** (C++, C#) – niestandardowe zachowanie operatorów dla klas użytkownika
- **Konstruktorów** – umożliwia różne sposoby inicjalizacji obiektu

Przeciążanie funkcji (C++)

- Język **C** nie obsługiwał przeciążania funkcji

```
#include <iostream>
using namespace std;

int divide(int a, int b) {
    return a / b;
}

double divide(double a, double b) {
    return a / b;
}

int main() {
    cout << "Int division: " << divide(10, 2) << endl;
    cout << "Double division: " << divide(10.0, 2.0) << endl;
    return 0;
}
```

- **C++** pozwala na zdefiniowanie wielu funkcji o tej samej nazwie, ale różniących się listą parametrów.
- Pozwala to na uproszczenie interfejsów – eliminuje konieczność stosowania różnych nazw dla podobnych operacji.

Przeciążanie metod

Na tej samej zasadzie działa przeciążanie metod w **Java**, **C++** i **C#**.

```
using namespace std;

class Calculator {
public:
    int divide(int a, int b) {
        return a / b;
    }
    double divide(double a, double b) {
        return a / b;
    }
};

int main() {
    Calculator calc;

    cout << calc.divide(7, 2) << endl;    // 3
    cout << calc.divide(7.0, 2.0) << endl; // 3.5

    return 0;
}
```

- Na podstawie listy parametrów kompilator musi być w stanie ustalić, którą metodę należy wywołać.

Przeciążanie a nadpisywanie

- **Przeciążanie (*Overloading*):** Ta sama nazwa metody, inna lista parametrów.
- **Nadpisywanie (*Overriding*):** Nowa implementacja metody w klasie dziedziczącej (np. `ToString`).

```
class Printer {  
    private string serialNumber;  
  
    // Konstruktor inicjalizujący numer seryjny  
    public Printer(string serialNumber) {  
        this.serialNumber = serialNumber;  
    }  
  
    // Przeciążona metoda Print dla liczb całkowitych  
    public void Print(int number) {  
        Console.WriteLine("Printing number: " + number);  
    }  
  
    // Przeciążona metoda Print dla tekstu  
    public void Print(string text) {  
        Console.WriteLine("Printing text: " + text);  
    }  
  
    // Nadpisana metoda ToString() zwracająca numer seryjny  
    public override string ToString() {  
        return "Printer Serial Number: " + serialNumber;  
    }  
}
```

Zasady przeciążania metod

Metody mają tę samą nazwę, ale muszą różnić się listą parametrów.

- Różnica może dotyczyć **liczby parametrów**.

```
class CharityDonation {  
    // Wpłata z domyślną kwotą  
    public void Donate() { ... }  
  
    // Wpłata z podaną kwotą  
    public void Donate(double amount) { ... }  
  
    // Wpłata z podaną kwotą i celem szczegółowym  
    public void Donate(double amount, string cause) { ... }  
}
```

Na podstawie liczby parametrów przy wywołaniu, kompilator rozstrzyga, którą z metod o tych samych nazwach wywołać.

```
CharityDonation myDonations = new CharityDonation();  
  
myDonations.Donate();                // Domyślna wpłata  
myDonations.Donate(50.0);            // Podana kwota  
myDonations.Donate(100.0, "Children's Fund"); // Podana kwota i cel
```

Zasady przeciążania metod

Metody mają tę samą nazwę, ale muszą różnić się listą parametrów.

- Różnica może dotyczyć **typu argumentów**.

```
class Range {  
    // Czy liczba należy do zakresu  
    public bool Contains(int value) { ... }  
  
    // Czy zakres zawiera inny zakres  
    public bool Contains(Range other) { ... }  
}
```

Jeśli liczba parametrów jest taka sama, różne muszą być ich typy.

```
Range range = new Range(2, 10);  
Range otherRange = new Range(3, 5);  
  
range.Contains(5);           // czy zawiera 5  
range.Contains(otherRange);  // czy zawiera [3,5]
```

Zasady przeciążania metod

Metody mają tę samą nazwę, ale muszą różnić się listą parametrów.

- Różnica może dotyczyć **kolejności argumentów** o różnych typach.

```
class BankAccount {  
    // Metoda do przelewu - kwota jako pierwszy parametr  
    public void Transfer(double amount, string accountNumber) { ... }  
  
    // Metoda do przelewu - konto jako pierwszy parametr  
    public void Transfer(string accountNumber, double amount) { ... }  
}
```

Podobnie jak poprzednio, kompilator na podstawie typu jest w stanie rozpoznać właściwą metodę.

```
BankAccount account = new BankAccount("ACC123");  
  
account.Transfer(150, "ACC456"); // Kwota -> Konto docelowe  
account.Transfer("ACC456", 150); // Konto docelowe -> Kwota
```


Po co przeciążamy metody?

Przeciążanie pozwala utrzymać **jedną abstrakcję operacji**, mimo że dane mogą mieć różną postać.

Przykład:

```
print(int)
print(string)
print(double)
```

To wszystko jest ta sama **operacja logiczna**.

Dzięki przeciążaniu:

- interfejs jest spójny
- kod jest czytelniejszy
- użytkownik klasy nie musi pamiętać wielu nazw metody

Błędy przeciążania metod

- **Nie wystarczy**, że metody różnią się **nazwą parametru**.

```
class Point {  
    private double x, y;  
  
    public void Set(double x) { // ❌ Błąd: różni się tylko nazwą parametru  
        this.x = x;  
    }  
  
    public void Set(double y) { // ❌ Błąd: różni się tylko nazwą parametru  
        this.y = y;  
    }  
}
```

Nazwa parametru nie wpływa na **sygnaturę metody**. Kompilator traktuje obie metody jako `Set(double)`.

A member 'Point.Set(double)' is already defined.

Na podstawie wywołania kompilator nie byłby w stanie określić o którą metodę chodzi.

```
Point point = new Point(0, 0);  
point.Set(3); // ?
```

Błędy przeciążania metod

- **Nie można** przeciążyć funkcji wyłącznie na podstawie **typu zwracanej wartości**.

```
class BankAccount {  
    private double balance;  
  
    public double GetBalance() { // ❌ Błąd: różni się tylko typem zwracanym  
        return balance;  
    }  
  
    public string GetBalance() { // ❌ Błąd: różni się tylko typem zwracanym  
        return $"{balance} PLN";  
    }  
}
```

Typ zwracany nie jest częścią sygnatury metody.

```
BankAccount account = new BankAccount();  
double balance = account.GetBalance();  
string balance = account.GetBalance();
```

Kompilator nie sprawdza, do jakiego typu przypisujemy wynik przed wywołaniem metody. Najpierw musi rozwiązać, którą wersję `GetBalance()` wywołać, zanim w ogóle przypisze wynik.

Błędy przeciążania metod

- Niejednoznaczność wyboru metody.

```
class Calculator {  
public:  
    int add(int a, double b) {  
        return a + (int)b;  
    }  
  
    double add(double a, int b) {  
        return a + b;  
    }  
};
```

Przy wywołaniu metody, kompilator próbuje dopasować argumenty do istniejących sygnatur, w razie potrzeby wykonując **konwersję typów**.

```
Calculator calc;  
cout << calc.add(3, 2); // ❌ Która metoda zostanie wywołana?
```

Jeśli istnieją równoważne konwersje, kompilator nie jest w stanie dokonać wyboru i zgłasza **błąd**.

```
error: call of overloaded 'add(int, int)' is ambiguous  
       candidate: 'int Calculator::add(int, double)'  
       candidate: 'double Calculator::add(double, int)'
```

Przeciążanie metod – dobre praktyki

- Spójność w działaniu – przeciążone metody powinny wykonywać podobne operacje i mieć intuicyjnie powiązane znaczenie.

✗ Źle – modyfikacja stanu lub zachowanie stanu

```
class DataHandler {
    int data;
public:
    // Sumowanie liczb
    void process(int value) {
        data += value;
    }

    // Zapis do pliku
    bool process(string filename) {
        ofstream file(filename);
        if(file.is_open()) {
            file << data << endl;
            file.close();
            return true;
        }
        return false;
    }
};
```

✗ Źle – niejednoznaczne nazwa

```
class Geometry {
public:
    // Obliczanie pola prostokąta
    int area(int width, int height) {
        return width * height;
    }

    // Obliczanie pola koła
    double area(int radius) {
        return 3.14159 * radius * radius;
    }
};
```

Przeciążanie metod – dobre praktyki

- Wspólna implementacja – aby uniknąć duplikacji kodu, warto delegować żądanie do innej z przeciążonych metod.

✓ Dobrze

```
class Greeter {  
    void greet() {  
        greet("Guest");  
    }  
  
    void greet(String name) {  
        greet(name, 18);  
    }  
  
    void greet(String name, int age) {  
        System.out.println("Hello " + name + ", you are " + age + " years old.");  
    }  
}
```

Celem przeciążania metod powinno być:

- Stworzenie bardziej intuicyjnego interfejsu programistycznego (API).
- Zapewnienie elastycznej interakcji.
- Zwiększenie czytelności kodu.

Alternatywa przeciążania - zmienna liczba argumentów

Możemy napisać metodę, która przyjmuje dowolną liczbę parametrów.

Java - `varargs ( ... )`

```
class Bank {  
    void sendMoney(String recipient, double... amounts) {  
        double total = 0;  
        for (double amount : amounts) {  
            total += amount;  
        }  
        System.out.println("Sending $" + total + " to " + recipient);  
    }  
}
```

Metoda `sendMoney()` przyjmuje jeden argument obowiązkowy (`String recipient`), a następnie dowolną liczbę argumentów typu `double`.

```
Bank bank = new Bank();  
bank.sendMoney("Alice", 100, 50.5, 20); // Sending $170.5 to Alice  
bank.sendMoney("Bob", 200);           // Sending $200 to Bob
```

Alternatywa przeciążania - zmienna liczba argumentów

Możemy napisać metodę, która przyjmuje dowolną liczbę parametrów.

C# - params

```
using System;

class Bank {
    public void SendMoney(string recipient, params double[] amounts) {
        double total = 0;
        foreach (double amount in amounts) {
            total += amount;
        }
        Console.WriteLine($"Sending ${total} to {recipient}");
    }
}
```

Pierwszy argument jest obowiązkowy, w kolejnych można podać dowolną liczbę kwot.

```
Bank bank = new Bank();
bank.SendMoney("Alice", 100, 50.5, 20); // Sending $170.5 to Alice
bank.SendMoney("Bob", 200);             // Sending $200 to Bob
```


Alternatywa przeciążania - zmienna liczba argumentów

Możemy napisać metodę, która przyjmuje dowolną liczbę parametrów.

C++ - `std::initializer_list`

```
#include <initializer_list>

class Bank {
public:
    void sendMoney(string recipient, initializer_list<double> amounts) {
        double total = 0;
        for (double amount : amounts) {
            total += amount;
        }
        cout << "Sending $" << total << " to " << recipient << endl;
    }
};
```

Kwoty są przekazywane jako lista.

```
Bank bank;
bank.sendMoney("Alice", {100, 50.5, 20}); // Sending $170.5 to Alice
bank.sendMoney("Bob", {200});             // Sending $200 to Bob
```

Alternatywa przeciążania - zmienna liczba argumentów

Możemy napisać metodę, która przyjmuje dowolną liczbę parametrów.

C++ - Variadic Template

```
class Bank {  
public:  
    template<typename... Args>  
    void sendMoney(string recipient, Args... amounts) {  
        double total = (amounts + ...); // Fold expression (C++17+)  
        cout << "Sending $" << total << " to " << recipient << endl;  
    }  
};
```

Obsługuje różne typy i jest bardziej wydajne (działa w czasie kompilacji).

```
Bank bank;  
bank.sendMoney("Alice", 100, 50.5, 20); // Sending $170.5 to Alice  
bank.sendMoney("Bob", 200);             // Sending $200 to Bob
```

Alternatywa przeciążania - parametry domyślne

Domyślne parametry pozwalają na określenie wartości domyślnych dla argumentów funkcji/metody. Dzięki temu można uniknąć przeciążania metod, gdy różne wersje różnią się tylko liczbą parametrów.

C++

```
class Greeter {  
public:  
    void greet(string name = "Guest", int age = 18) {  
        cout << "Hello " << name << ", you are " << age << " years old.\n";  
    }  
};
```

C#

```
class Greeter {  
    void Greet(string name = "Guest", int age = 18) {  
        Console.WriteLine($"Hello {name}, you are {age} years old.");  
    }  
}
```

Wywołanie:

```
greeter.greet();           // Hello Guest, you are 18 years old.  
greeter.greet("Alice");    // Hello Alice, you are 18 years old.  
greeter.greet("Bob", 25);  // Hello Bob, you are 25 years old.
```

Alternatywa przeciążania – parametry domyślne

Ograniczenia

- Parametry domyślne muszą być na końcu listy parametrów.

```
int Add(int a = 5, int b) { return a + b; } // ❌ Błąd  
int Add(int a, int b = 10) { return a + b; } // ✅ OK
```

- Również przy wywołaniu – możemy pominąć argumenty na końcu listy.

```
int Add(int a = 5, int b = 10) { return a + b; }  
  
Console.WriteLine(Add());      // 15  
Console.WriteLine(Add(3));     // 13  
Console.WriteLine(Add(3, 4));  // 7
```

- Parametry domyślne muszą być wartościami stałymi

```
int x = 10;  
int Add(int a, int b = x) { return a + b; } // ❌ Błąd
```

- Java** nie obsługuje parametrów domyślnych.

Parametry nazwane (C#)

- **Parametry nazwane** w C# pozwalają na przekazywanie argumentów do metody jawnie określając ich nazwy.

```
void PrintMessage(string message, int count) {  
    for (int i = 0; i < count; i++) {  
        Console.WriteLine(message);  
    }  
}
```

Standardowe wywołanie nie jest jasne – co oznacza liczba „3”?

```
PrintMessage("Hello", 3); // ?
```

Parametry nazwane zwiększają czytelność kodu.

```
PrintMessage("Hello", count: 3);
```

Umożliwiają również przekazanie argumentów w dowolnej kolejności.

```
PrintMessage(count: 2, message: "Goodbye");
```

Parametry nazwane (C#)

- Parametry nazwane łączą się z parametrami opcjonalnymi (domyślnymi).

```
void LogMessage(  
    string message = "System is operational",  
    string type = "INFO",  
    string icon = "i ")  
{  
    Console.WriteLine($"{icon} [{type}] {message}");  
}
```

Parametry nazwane pozwalają na elastyczne wywołanie `LogMessage()`, zmieniając tylko wybrane argumenty i podając je w dowolnej kolejności.

```
// Wywołanie domyślne ⇒ i [INFO] System is operational  
LogMessage();  
  
// Wywołanie z własnym komunikatem ⇒ i [INFO] User logged in  
LogMessage("User logged in");  
  
// Wywołanie z parametrami nazwanymi ⇒ ! [ALERT] Low disk space  
LogMessage(type: "ALERT", message: "Low disk space", icon: "!");  
  
// Wywołanie z podaniem tylko części parametrów ⇒ Wi [INFO] Running in wireless mode  
LogMessage("Running in wireless mode", icon: "Wi");
```

Metody *read-only* w C++

W C++ oznaczenie metody jako `const` sprawia, że nie może ona modyfikować stanu obiektu.

```
class Example {  
private:  
    int value;  
public:  
    Example(int v) : value(v) {}  
  
    // Metoda const - nie zmienia stanu obiektu  
    int getValue() const {  
        //value++;    // ❌ Błąd kompilacji - próba zmiany stanu  
        return value; // ✅ Działa, nie zmieniamy stanu  
    }  
  
    // Zwykła metoda - może modyfikować stan obiektu  
    void setValue(int v) {  
        value = v;  
    }  
};  
  
const Example obj(10);  
cout << obj.getValue(); // ✅ Działa, bo getValue() jest const  
obj.setValue(20); // ❌ Błąd kompilacji - setValue() nie jest const
```

Dobra zasada: oznaczajmy jako `const` wszystkie metody niemodyfikujące.

Metody *read-only* w C++

- Dozwolone jest przeciążanie metod na podstawie tego, czy są `const` czy też nie.

```
class Example {
private:
    int value;
public:
    Example(int v) : value(v) {}

    // Metoda const - nie zmienia stanu obiektu
    int getValue() const {
        return value;
    }

    // Metoda nie-const - zmienia stan
    int getValue() {
        value++;
        return value;
    }
};

const Example obj1(5);
Example obj2(10);

cout << obj1.getValue() << endl; // 5
cout << obj2.getValue() << endl; // 11
```


Metody z parametrem `const` w C++

- Oznaczając parametr jako `const`, zapobiegamy jego modyfikacji wewnątrz metody/funkcji.

```
void showValue(const Example& example) {  
    cout << example.getValue() << endl;  
}
```

- Działa to tak samo jak pole lub zmienna `const`.
- Na obiekcie `const` można wykonać jedynie operacje `const`.
- Przeważnie używane z parametrami rodzaju *referencja na obiekt* - zapobiega to kopiowaniu.
- Łączy się to z przeciążaniem - wariant bez `const` wywoła metodę bez `const`.

```
void showValue(Example& example) {  
    cout << example.getValue() << endl;  
}
```

- `final` w **Java** i `readonly` w **C#** nie zapewniają tej funkcjonalności - przed zmianą chroniona jest jedynie referencja, a nie stan obiektu.

```
void change(final Example example) {  
    example = new Example(20); // ❌ - niedozwolone  
    example.setValue(20);      // ✅ - dozwolone  
}
```

Modyfikatory parametrów z C# – `ref`

Domyślnie parametry przekazywane są przez wartość.

- W przypadku typów wartościowych (np. `int`), przekazywana jest *kopia wartości*.

```
void Foo(int x) { x = 10; }

int a = 5;
Foo(a); // a = 5
```

- W przypadku typów referencyjnych (obiekty), przekazywana jest *kopia referencji*.

```
void Foo(Point p) {
    p = new Point(3, 4);
}

Point p = new Point(5, 2);
Foo(p); // p = (5, 2)
```

```
void Goo(Point p) {
    p.x = 3;
    p.y = 4;
}

Point p = new Point(5, 2);
Goo(p); // p = (3, 4)
```

- Modyfikator `ref` pozwala przekazać parametr przez referencję (argument musi zostać zainicjowany przed wywołaniem metody).

```
void Foo(ref int x) { x = 10; }

int a = 5;
Foo(ref a); // a = 10
```

```
void Foo(ref Point p) {
    p = new Point(3, 4);
}

Point p = new Point(5, 2);
Foo(ref p); // p = (3, 4)
```

Modyfikatory parametrów z C# – out

- Przekazywanie przez `out` – zwracanie wyniku
 - Nie wymaga inicjalizacji argumentu przed wywołaniem.
 - Metoda musi przypisać wartość do argumentu.

```
void Foo(out int x) { x = 10; }  
int a;  
Foo(out a); // a = 10
```

Przykład – `TryParse()` :

```
string input = "123";  
if(int.TryParse(input, out int result))  
{  
    Console.WriteLine(result);  
}  
else  
{  
    Console.WriteLine("Nieprawidłowy format liczby");  
}
```

- Zmienna przekazywana jako drugi parametr może być zadeklarowana w liście argumentów:
`int.TryParse(input, out int result)`.

Modyfikatory parametrów z C# - `in`, `ref readonly`

- Przekazywanie przez `in` (tylko do odczytu)
 - Gwarantuje, że wartość nie zostanie zmodyfikowana.
 - Argument przekazywany przez referencję (być może na tymczasową wartość).
 - Nie wymaga podania modyfikatora w miejscu wywołania.

```
void Foo(in int x) {  
    Console.WriteLine(x);  
}
```

- Przekazywanie przez `ref readonly`
 - Wartość tylko do odczytu.
 - Kompilator spodziewa się referencji na zmienną.
 - Nie wymaga podania modyfikatora w miejscu wywołania.

```
void Foo(ref readonly Point p) {  
    Console.WriteLine($"({p.x}, {p.y})");  
}
```

- Składowe klasy nie mogą mieć sygnatur, które różnią się tylko `ref`, `ref readonly`, `in` lub `out`.

Przeciążanie konstruktorów

Przeciążanie konstruktorów

- Zasady przeciążania metod czy korzystania z parametrów domyślnych dotyczą również konstruktorów.

```
class Rectangle {  
    private double width;  
    private double height;  
  
    // Konstruktor domyślny - tworzy prostokąt o wymiarach 0x0  
    public Rectangle()  
    {    this.width = this.height = 0; }  
  
    // Konstruktor z jednym parametrem - kwadrat o boku 'size'  
    public Rectangle(double size)  
    {    this.width = this.height = size; }  
  
    // Konstruktor pełny - przyjmuje szerokość i wysokość  
    public Rectangle(double width, double height)  
    {    this.width = width; this.height = height; }  
}
```

- Pozwala to na tworzenie obiektów na różne sposoby, np. z domyślnymi wartościami lub określonymi argumentami.

```
Rectangle r1 = new Rectangle();    // Domyślny (0x0)  
Rectangle r2 = new Rectangle(5);   // Kwadrat 5x5  
Rectangle r3 = new Rectangle(4, 6); // Prostokąt 4x6
```

Delegowanie konstruktorów

- Aby uniknąć powielania kodu, dobrą praktyką jest delegowanie konstrukcji obiektu do innego konstruktora.

```
class Rectangle {  
    private double width;  
    private double height;  
  
    // Konstruktor domyślny - tworzy prostokąt o wymiarach 0x0  
    public Rectangle() {  
        this(0); // wywołujemy konstruktor z jednym parametrem  
    }  
  
    // Konstruktor z jednym parametrem - kwadrat o boku 'size'  
    public Rectangle(double size) {  
        this(size, size); // wywołujemy konstruktor dwuparametrowy  
    }  
  
    // Konstruktor pełny - przyjmuje szerokość i wysokość  
    public Rectangle(double width, double height) {  
        this.width = width;  
        this.height = height;  
    }  
}
```

- W **Javie** używa się do tego słowa kluczowego `this(...)` - musi być pierwszą instrukcją konstruktora.

Delegowanie konstruktorów w C++ i C#

- W **C++** i **C#** wywołanie innego konstruktora klasy wygląda nieco inaczej.

C++

```
Rectangle(double size) : Rectangle(size, size) {  
    // ciało może być puste lub zawierać dodatkowe instrukcje  
}
```

C#

```
public Rectangle(double size) : this(size, size) {  
    // ciało może być puste lub zawierać dodatkowe instrukcje  
}
```

Jeśli wartości domyślne są stałe, można ograniczyć liczbę przeciążonych konstruktorów dzięki parametrom domyślnym.

```
public Rectangle(double width = 0, double height = 0) {  
    this.width = width;  
    this.height = height;  
}
```


Konstruktor kopiujący

W języku **C++** specjalną rolę ma **konstruktor kopiujący**

- Umożliwia tworzenie **nowego obiektu** jako **kopii istniejącego**.
- Przyjmuje jako parametr inny obiekt tej samej klasy.
- Jego rolą jest zainicjować aktualną instancję (`this`) na dokładną kopię przekazanego obiektu.

```
class Rectangle {  
    int width, height;  
public:  
    Rectangle(int w, int h) : width(w), height(h) {}  
    Rectangle(const Rectangle& other) {  
        width = other.width;  
        height = other.height;  
    }  
};
```

- Jest wywoływany w sytuacji, gdy tworzymy obiekt inicjując go wartością innego obiektu.

```
Rectangle a(10, 20);  
Rectangle b = a; // wywołanie konstruktora kopiującego
```

- Odpowiada to temu:

```
Rectangle b(a);
```

Konstruktor kopiujący w C++

- Konstruktor kopiujący jest również wywoływany, gdy przekazujemy parametr przez **wartość**.

```
void display(Rectangle rect) {  
    ...  
}
```

- To dlatego obiekt jest przekazywany do konstruktora kopiującego **przez referencję** (w przeciwnym razie dochodziłoby do stworzenia kopii i rekurencyjnego wywołania konstruktora kopiującego).

```
Rectangle(const Rectangle& other) {  
    ...  
}
```

- Jeśli przypisujemy wartość do obiektu, który już istnieje, konstruktor kopiujący się **nie wywołuje**. Poniższy zapis jest przykładem wywołania **operatora przypisania**.

```
Rectangle a();  
Rectangle b(10, 20);  
  
a = b;
```

Domyślny konstruktor kopiujący w C++

- Jeśli sami nie zdefiniujemy konstruktora kopiującego, kompilator utworzy go za nas – w postaci domyślnej.
- Kopiuje on wszystkie wartości pól:

```
Rectangle(const Rectangle& other) {  
    width = other.width;  
    height = other.height;  
}
```

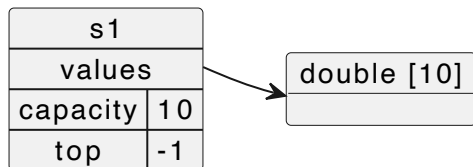
- Taka wersja jest wystarczająca, jeśli wszystkie pola są typu wartościowego (jak w klasie `Rectangle`), ale rodzi problemy, jeśli są typu wskaźnikowego (jak w klasie `Stack`):

```
class Stack {  
    double* values;  
    int capacity;  
    int top;  
public:  
    Stack(const Stack& other) {  
        values = other.values;    // ❌ UWAGA: przepisanie wskaźnika  
        capacity = other.capacity;  
        top = other.top;  
    }  
}
```

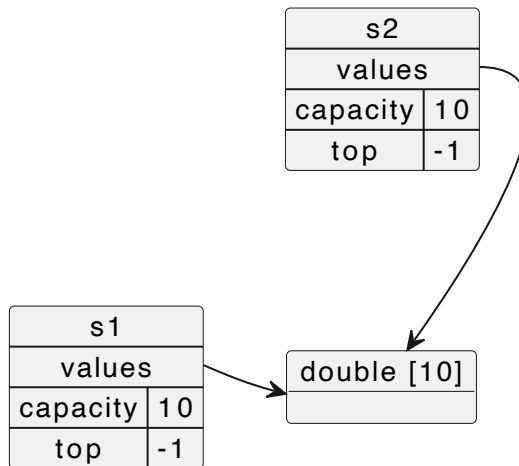
Domyślny konstruktor kopiujący w C++

- Domyślny konstruktor kopiujący wykonuje **płytką kopię** (*shallow copy*) obiektu.
- Skopiowaliśmy wartość wskaźnika, ale oba wskazują na tę samą pamięć.

```
Stack s1(10);
```



```
Stack s2 = s1;
```



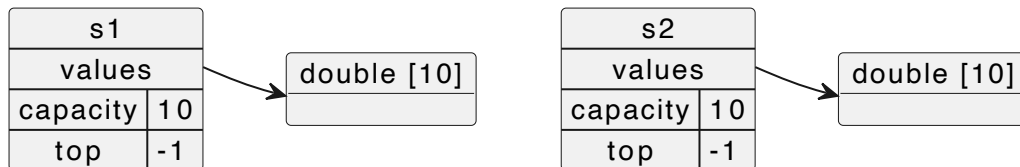
- **Problem:** dwa obiekty korzystają ze wspólnej pamięci.

Poprawny konstruktor kopiujący w C++

```
class Stack {  
    double* values;  
    int capacity;  
    int top;  
  
public:  
    // Konstruktor kopiujący  
    Stack(const Stack& other) : capacity(other.capacity), top(other.top) {  
        values = new double[capacity]; // nowa tablica!  
        for (int i = 0; i <= top; ++i) {  
            values[i] = other.values[i]; // kopiujemy dane  
        }  
    }  
};
```

- Ta wersja kopiuje dane, a nie tylko wskaźnik – powstaje **kopia głęboka**.

```
Stack s1(10);  
Stack s2 = s1;
```



Przeciążanie operatorów

Przeciążanie operatorów

- **C++** i **C#** umożliwiają przeciążanie operatorów.
- Pozwala to redefiniować działanie operatorów (`+` , `-` , `+=` , `=` , `==` , `[]` itp.) dla obiektów naszych klas.
- Operator to metoda o specjalnej sygnaturze, np. `operator+( )` .
- Dzięki temu, operacje na obiektach własnych typów mogą wyglądać bardziej intuicyjnie, np. `A + B` zamiast `A.Add(B)` .
- Ułatwia to pracę z obiektami, poprawia czytelność kodu i umożliwia definiowanie własnych typów numerycznych (np. `Decimal` , `Matrix` , `Complex`).
- Warto korzystać z możliwości przeciążania operatorów, gdy operacja ma naturalne znaczenie matematyczne (np. `+` dla macierzy), a obiekty powinny pełnić rolę typów prostych.

Przeciążanie operatorów w C++

- Przykład – klasa `Vector2D`, reprezentująca wektor w dwuwymiarowej przestrzeni (x, y) .

```
class Vector2D {  
    double x, y;  
  
public:  
    // Konstruktor  
    Vector2D(double x = 0, double y = 0) : x(x), y(y) {}  
};
```

Chcemy móc zapisywać operacje na wektorach w sposób naturalny, jak na typach prostych:

```
Vector2D v1(3, 4);  
Vector2D v2(1, 2);  
  
// Sumowanie wektorów  
Vector2D sum = v1 + v2;
```

Wymaga to przeciążenia operatora `+`. Możemy go przeciążyć

- jako jednoargumentową metodę: `Vector2D operator+(Vector2D)`
- jako dwuargumentową funkcję globalną: `Vector2D operator+(Vector2D, Vector2D)`

Przeciążanie operatora `+` w C++

- Za wywołaniem `sum = v1 + v2` kryje się `sum = v1.operator+(v2)`.
- Operator jest wywoływany z lewego argumentu, natomiast prawy jest przekazany parametrem.
- Metoda nie zmienia obiektu, z którego jest wywołana (ani parametru), zwraca natomiast jako wynik nowy obiekt, który zostanie zapisany w `sum`.

```
class Vector2D {  
    double x, y;  
  
public:  
    // Konstruktor  
    Vector2D(double x = 0, double y = 0) : x(x), y(y) {}  
  
    // Przeciążenie operatora +  
    Vector2D operator+(const Vector2D& other) const {  
        return Vector2D(x + other.x, y + other.y);  
    }  
};
```

- Przekazanie parametru jako `const&` zapobiega kopiowaniu i zapewnia, że metoda go nie zmieni.
- Metoda jest *read-only* (`const`) co pozwoli na jej wywołanie ze stałych referencji
- Wynik zwracamy (przez kopię) jako nowy obiekt (automatyczny).
- W podobny sposób działają inne operatory binarne (`-`, `*`, `^`, `%`, `&`).

Przeciążanie operatora `=` w C++

- Gdy wykonujemy operację `sum = v1 + v2` dodawanie to tylko prawa strona równania, pozostaje jeszcze `sum = ...`, czyli operator przypisania.

```
class Vector2D {
    double x, y;

public:
    // Konstruktor
    Vector2D(double x = 0, double y = 0) : x(x), y(y) {}

    // Przeciążenie operatora =
    Vector2D& operator=(const Vector2D& other) {
        x = other.x;
        y = other.y;

        // Zwracamy referencję na samego siebie
        return *this;
    }
};
```

- Przepisujemy od siebie (lewego argumentu) wartość prawego argumentu.
- Jako wynik zwracamy lewy argument, co pozwoli na łączenie przypisań: `v1 = v2 = v3;`.
- **Uwaga:** przypisanie podczas inicjalizacji (`Vector2D v3 = v1;`) jest wywołaniem **konstruktora kopiującego**, a nie operatora przypisania.

Automatyczny operator `=` w C++

- Jeśli nie zdefiniujemy operatora przypisania, kompilator wygeneruje nam domyślny – wykonujący przepisanie wszystkich pól:

```
Vector2D& operator=(const Vector2D& other) {  
    x = other.x;  
    y = other.y;  
  
    // Zwracamy referencję na samego siebie  
    return *this;  
}
```

- Jest to wystarczające, jeśli wszystkie pola są typu wartościowego (jak w klasie `Vector2D`), ale rodzi problemy, jeśli są typu wskaźnikowego (jak w klasie `Stack`):

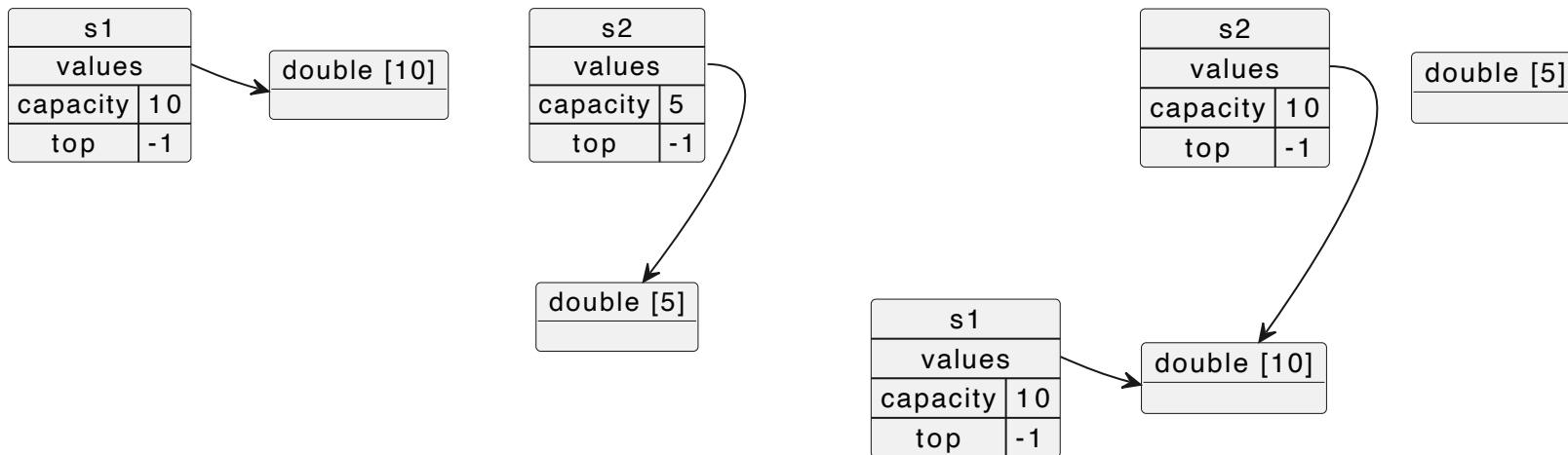
```
class Stack {  
    double* values;  
    int capacity;  
    int top;  
public:  
    Stack& operator=(const Stack& other) {  
        values = other.values;    // ❌ UWAGA: przepisanie wskaźnika  
        capacity = other.capacity;  
        top = other.top;  
        return *this;  
    }  
}
```

Domyślny operator `=` w C++

- Domyślny operator `=` wykonuje **płytką kopię** (*shallow copy*) obiektu.
- Skopiowaliśmy wartość wskaźnika, ale oba wskazują na tę samą pamięć.

```
Stack s1(10), s2(5);
```

```
s2 = s1;
```



- **Problem 1:** dwa obiekty korzystają ze wspólnej pamięci.
- **Problem 2:** nie zwolniliśmy prawidłowo tablicy obiektu `s2`.

Nie jest to problemem w **C#** i **Java**, gdzie przypisanie `s2 = s1` zawsze kopiuje referencję, a nie obiekt.

Poprawny operator `=` w C++

```
class Stack {
    double* values;
    int capacity;
    int top;

public:
    // Operator przypisania
    Stack& operator=(const Stack& other) {
        if (this != &other) { // Unikamy przypisania do samego siebie (a = a)
            delete[] values; // Zwolnienie starej pamięci
            // Kopiowanie danych
            capacity = other.capacity;
            top = other.top;
            values = new double[capacity]; // Alokacja nowej pamięci
            for (int i = 0; i <= top; i++) {
                values[i] = other.values[i]; // Kopiowanie elementów
            }
        }
        return *this;
    }
};
```

- Ta wersja kopiuje dane, a nie tylko wskaźnik.

Przeciążanie operatora `+=` w C++

- Aby móc dodać jeden obiekt do drugiego bez tworzenia nowego obiektu, należy przeciążyć operator `+=`.

```
v1 = v1 + v2; // Tworzy nowy obiekt tymczasowy  
v1 += v2;     // Modyfikuje v1 bez tworzenia nowego obiektu
```

Łączy on cechy operatora `+` i operatora `=`.

```
class Vector2D {  
    double x, y;  
  
public:  
    Vector2D(double x = 0, double y = 0) : x(x), y(y) {}  
  
    // Operator += (modyfikuje istniejący obiekt)  
    Vector2D& operator+=(const Vector2D& other) {  
        x += other.x;  
        y += other.y;  
        return *this;  
    }  
};
```

Przeciążanie operatora `==` w C++

- Przeciążenie operatora `==` pozwoli na naturalne porównywanie dwóch wektorów.

```
class Vector2D {  
    double x, y;  
  
public:  
    Vector2D(double x = 0, double y = 0) : x(x), y(y) {}  
  
    // Operator porównania  
    bool operator==(const Vector2D& other) const {  
        return (x == other.x) && (y == other.y);  
    }  
};
```

Dzięki temu porównywanie wektorów wygląda i działa tak samo, jak porównywanie typów wbudowanych (`int` , `double`).

```
Vector2D v1(3, 4), v2(3, 4), v3(1, 2);  
  
cout << (v1 == v2 ? "v1 i v2 są równe" : "v1 i v2 są różne") << endl;  
cout << (v1 == v3 ? "v1 i v3 są równe" : "v1 i v3 są różne") << endl;
```

- Uwaga:** porównywanie wartości zmiennoprzecinkowych nie jest bezpieczne, w praktyce lepiej porównywać z tolerancją, np. `std::abs(x - other.x) < EPS` (gdzie np. `EPS = 1e-9`)

Przeciążanie operatora << w C++

- Aby móc wypisywać obiekt `Vector2D` na standardowe wyjście tak samo, jak wartości typów wbudowanych (`cout << v1`), należy przeciążyć operator `<<` (przekierowania).
- Nie jest to metoda klasy `Vector2D` , gdyż po lewej stronie znajduje się obiekt klasy `ostream` - przeciążamy go zatem jako funkcję dwuparametrową: `ostream& operator<<(ostream& os, const Vector2D& vec)` .
- Ten operator korzysta z prywatnych zmiennych klasy `Vector2D` - aby to zadziałało, klasa `Vector2D` musi zadeklarować zaprzyjaźnienie: `friend ostream& operator<<(ostream&, const Vector2D&);` .

```
class Vector2D {  
    ...  
public:  
    // Przeciążenie operatora <<  
    friend ostream& operator<<(ostream& os, const Vector2D& vec) {  
        os << "(" << vec.x << ", " << vec.y << ")"; // UWAGA: wypisanie na os a nie cout!  
        return os; // Zwracamy os, aby umożliwić łączenie przekierowania: cout << v1 << endl;  
    }  
};
```

- Tak przygotowany operator pozwoli też wypisać `Vector2D` do pliku (`ofstream`) czy innego strumienia wyjściowego.

Przeciążanie operatora `++` w C++

- Operatory inkrementacji (i dekrementacji) można przeciążać w wersji post- i pre-.

```
class Integer {  
    int value;  
  
public:  
    // Konstruktor  
    Integer(int v = 0) : value(v) {}  
  
    // Preinkrementacja (++obj)  
    Integer& operator++() {  
        ++value; // Zwiększa wartość  
        return *this; // Zwraca zmodyfikowany obiekt  
    }  
  
    // Postinkrementacja (obj++)  
    Integer operator++(int) {  
        Integer temp = *this; // Tworzy kopię aktualnego obiektu  
        ++value; // Zwiększa wartość  
        return temp; // Zwraca kopię sprzed zmiany  
    }  
};
```

- Preinkrementacja jest bardziej efektywna (nie wymaga tworzenia kopii obiektu).
- Postinkrementacja jest przydatna, gdy potrzebujemy wartości przed zmianą.

Przeciążanie operatora `[]` w C++

- Przeciążenie operatora `[]` pozwala na dostęp do elementów obiektu tak, jakby był tablicą.

```
class Array {  
    int* data;      // Dynamiczna tablica  
    int size;      // Rozmiar tablicy  
  
public:  
    ...  
    // Przeciążenie operatora [] (dla odczytu i zapisu)  
    int& operator[](int index) {  
        return data[index]; // Zwraca referencję, umożliwiając modyfikację  
    }  
  
    // Stała wersja operatora [] (dla odczytu w obiektach const)  
    const int& operator[](int index) const {  
        return data[index];  
    }  
};
```

Daje to możliwość odczytu i zapisy wartości przez operatora `[]`.

```
Array arr(5);  
  
arr[0] = 10;  
cout << "Element at index 0: " << arr[0] << endl;
```

Przeciążanie operatora () w C++

- Przeciążenie operatora funkcyjnego () pozwala traktować obiekty klasy jak funkcje.

```
class Accumulator {
    int sum; // Przechowuje sumę

public:
    // Konstruktor inicjalizujący sumę na 0
    Accumulator() : sum(0) {}

    // Operator () z parametrem - dodaje wartość do sumy
    void operator()(int value) { sum += value; }

    // Operator () bez parametru - zwraca aktualną sumę
    int operator()() const { return sum; }
};
```

- Akumulator może być traktowany jako funkcja, ale zapamiętuje wartości, pozwalając na intuicyjny odczyt sumy.

```
Accumulator acc; // Tworzymy obiekt akumulatora

acc(5); // Dodajemy 5
acc(10); // Dodajemy 10
acc(3); // Dodajemy 3

cout << "Current sum: " << acc() << endl; // Wywołanie bez argumentów - zwraca sumę
```

Przeciążanie operatora rzutowania w C++

- W C++ można przeciążyć operator rzutowania, aby umożliwić konwersję obiektu klasy na typ wbudowany, np. `int`.

```
class Integer {  
    int value;  
  
public:  
    // Konstruktor  
    Integer(int v) : value(v) {}  
  
    // Przeciążenie operatora rzutowania na int  
    operator int() const {  
        return value; // Zwraca przechowywaną wartość jako int  
    }  
};
```

Pozwala to traktować obiekty naszej klasy tak, jakby były zwykłymi liczbami.

```
Integer num(42); // Tworzymy obiekt Integer  
  
// Automatyczna konwersja obiektu Integer na int  
int x = num;
```

C++ pozwala na przeciążenie większości operatorów (poza `.`, `!`, `::`)

Przeciążanie operatorów w C#

- Język **C#** jest bardziej restrykcyjny pod względem przeciążania operatorów (tylko część operatorów możemy bez przeszkód przeciążyć).
- Operatory muszą być metodami statycznymi.

```
class Vector2D {  
    public double X { get; }  
    public double Y { get; }  
  
    public Vector2D(double x, double y) { X = x; Y = y; }  
  
    // Przeciążenie operatora +  
    public static Vector2D operator +(Vector2D a, Vector2D b) {  
        return new Vector2D(a.X + b.X, a.Y + b.Y);  
    }  
}
```

- Możemy dodawać wektory przy pomocy operatora `+`.

```
Vector2D v1 = new Vector2D(3, 4);  
Vector2D v2 = new Vector2D(1, 2);  
  
// Dodawanie wektorów  
Vector2D sum = v1 + v2;
```

- Podobnie przeciąża się większość pozostałych operatorów niemodyfikujących (`-`, `|`, `>>`, `!`).

Operator `=` w C#

- Operator przypisania **nie może** być w C# przeciążony.
- Przypisuje on jedynie referencje.

```
Vector2D v1 = new Vector2D(3, 4);  
Vector2D v2 = new Vector2D(1, 2);  
  
v2 = v1;
```

- Nie możemy też jawnie przeciążać operatorów typu `+=`.
- Ale jeśli przeciążyliśmy `+`, możemy również używać `+=` (`a += b` jest synonimem `a = a + b`).

```
v2 += v1;
```

Przeciążanie operatorów `==` i `!=` w C#

- Operatory relacji **muszą** być przeciążane **parami** (tzn. jeśli przeciążamy operator `<`, powinniśmy też przeciążyć `>`).

```
class Vector2D {  
    // Przeciążenie operatora ==  
    public static bool operator ==(Vector2D a, Vector2D b) {  
        // Porównujemy wartości współrzędnych  
        return a.X == b.X && a.Y == b.Y;  
    }  
  
    // Przeciążenie operatora != (musi być zgodne z ==)  
    public static bool operator !=(Vector2D a, Vector2D b) {  
        return !(a == b);  
    }  
}
```

Teraz możemy porównywać wektory.

```
Vector2D v1 = new() { X = 3, Y = 4 };  
Vector2D v2 = new() { X = 3, Y = 4 };  
Vector2D v3 = new() { X = 1, Y = 2 };  
  
Console.WriteLine(v1 == v2); // true  
Console.WriteLine(v1 == v3); // false  
Console.WriteLine(v1 != v3); // true
```

Przeciążanie operatorów rzutowania w C#

- Operatory rzutowania mogą być przeciążane w dwóch wersjach:
 - jawnej (*explicit*)** – jeśli może powodować utratę danych
 - niejawnej (*implicit*)** – gdy rzutowanie jest bezpieczne

```
class DoubleValue {  
    public double Value { get; init; }  
    public DoubleValue(double value) { Value = value; }  
  
    // Rzutowanie NIEJAWNE na double (implicit) – bezpieczne, bez utraty danych  
    public static implicit operator double(DoubleValue dv) {  
        return dv.Value;  
    }  
    // Rzutowanie JAWNE na int (explicit) – może utracić część ułamkową  
    public static explicit operator int(DoubleValue dv) {  
        return (int)dv.Value; // Rzutowanie double na int (ucięcie części dziesiętnej)  
    }  
}
```

Pozwala to na konwersję między niestandardowymi typami a typami wbudowanymi.

```
DoubleValue dv = new DoubleValue(42.7);  
  
double d = dv; // Implicit (niejawne) rzutowanie na double – nie wymaga jawnego castowania  
int i = (int)dv; // Explicit (jawne) rzutowanie na int – wymaga castowania (int)
```


Indeksery w C#

- W **C#** nie można przeciążać operatora `[]`, ale można zdefiniować pełniący tę samą rolę **indexer**.
- Pozwala na traktowanie klasy jako tablicy i dostęp do elementów przy pomocy `[]`.

```
class Array {  
    private int[] data; // Tablica przechowująca elementy  
    public Array(int size) { data = new int[size]; } // Konstruktor  
  
    // Indeksery umożliwiające dostęp do elementów za pomocą []  
    public int this[int index] {  
        get {  
            return data[index];  
        }  
        set {  
            data[index] = value;  
        }  
    }  
}
```

- Podobnie jak właściwości, składa się z *gettera* i *settera*, używanych przy pobieraniu bądź ustawianiu wartości.

```
Array arr = new Array(5);  
  
arr[1] = 20;  
Console.WriteLine($"Element at index 1: {arr[1]}"); // 20
```